Measurement Invariance Power App

Shiny app and R functions for computing analytic power of chi-square difference tests of **scale-level** (parameter-total) vs **item-level** measurement invariance in multiple-group one-factor CFA.

Scale-level is also called *composite-level*: the constraint is equality of the parameter totals across groups, not equality of the individual item parameters.

Run the app

```r
shiny::runApp("path/to/ShinyApp")
```

Requires: `lavaan`, `shiny`, `bslib`, `ggplot2`, `DT`.

Files

File	Contents
`app.R`	Shiny UI/server only (no statistics live here)
`R/models.R`	Population specification (`default_params`, `apply_noninvariance`, `build_structured_params`, `structured_sweep`, `pop_moments`) and lavaan model generation for any number of items p and groups G (`make_item_model`, `make_scale_model`)
`R/power.R`	`mi_power()` / `mi_power_sweep()`: MacCallum-Browne-Cai (2006) analytic power for metric/scalar/strict tests, both approaches, returned as tidy data frames
`R/plots.R`	`plot_power()`: ggplot2 power curves, one facet per test. Solid = scale-level, dashed = item-level, both approaches in the same facet so they stay comparable. When more than one condition is overlaid, the conditions are shades within each approach's hue (blue = scale-level, orange = item-level, darkest = the first condition in the sweep) and each curve is labelled with its value at the right-hand tip, so the plot stays readable with many conditions
`validation/test_app.R`	End-to-end checks of the app itself: UI structure, every sidebar control, the contents of each tab, the plot, the downloads, and the error paths. Run: `Rscript validation/test_app.R` (133 checks, about 45 seconds)

Scripting the functions directly

```r
invisible(lapply(list.files("R", full.names = TRUE), source))
library(lavaan)

# k = 1..7 non-invariant loadings, +.1 each, on p = 8 items in 2 groups
conds <- structured_sweep(
  list(p = 8, n_groups = 2, type = "loadings", n_ni = 3, magnitude = .1),
  sweep_var = "n_ni", values = 1:7)
res <- mi_power_sweep(conds, tests = "metric")
plot_power(res$curves)
res$details   # population RMSEA, test df, N needed for 80% power
```

The app always places non-invariance on the last k items, with k at most p - 1: item 1 stays invariant because it anchors the latent scale in the analysis models. Because the remaining items are exchangeable, which items carry the difference does not affect the fit statistics or the power.

The test matching the manipulated parameter class always has power above alpha. The one cross-class effect is that non-invariant loadings also give the strict test non-zero population misfit, even when the population residual variances are equal across groups: the metric and scalar models constrain the unequal loadings to be equal, so the estimated residual variances differ across groups in those models, and the strict-level equality constraint then adds misfit. Non-invariant intercepts have no such effect, because intercepts enter only the mean structure. So with loadings alone manipulated, the metric and strict facets both rise above alpha and only the scalar facet stays flat at alpha.

Method notes

- Noncentrality is computed from the population minimum discrepancy: `nonc = (N_total - G) * (F0_constrained - F0_baseline)`, which for G = 2 is numerically identical to `n * df * rmsea^2` but is correct for any G and independent of lavaan's multi-group RMSEA convention. Baselines are one invariance level lower (configural -> metric -> scalar -> strict), fit with the same approach.
- Test df are taken from the fitted models rather than assumed. The item-level metric test has (p - 1)(G - 1) df, which is p - 1 for two groups; the scale-level metric test has G - 1 df.
- Scale-level models for G groups impose G - 1 total-equality constraints per parameter class (plus the first-intercept anchor for scalar).
- Non-invariance can affect the last m groups (1 <= m <= G - 1), each receiving the same differences; the remaining groups equal group 1.
- "Concentration" redistributes a fixed total difference over the non-invariant items without changing that total: the largest item's share is 1/k + concentration * (1 - 1/k), the rest is split evenly over the other k - 1 items. 0 is the even split and 1 puts everything on one item, so at concentration = 1 the k-item condition reproduces a single-item condition carrying the whole total, that is a per-item difference of k * delta rather than delta. Item-level power rises as the total is concentrated while scale-level power is unchanged.
- Population latent variance is 1 and latent mean 0 in all groups; ad-hoc effect sizes (beta_spurious, d_spurious) are intentionally omitted.